Activity 5 – Evaluation

Introduction

The Password Security Rating program is a console application for staff at a data-storage company. It accepts a password and reports whether its length is valid, the points breakdown, total score and security rating. It helps staff judge password strength consistently.

Review of Requirements

- Enter a password: implemented using console input.

- Identify too short/too long: lengths below 8 and above 15 produce clear validation messages and stop scoring.

- Allowed characters: only lowercase and uppercase letters, digits and `! % & + =` are accepted.

- Points breakdown: counts and scores for each character class, length bonus and category-only deductions are displayed.

- Total score and rating: total is calculated and mapped to the five supplied rating bands.

- Robust, user-friendly, conventional code: validation, descriptive names, functions, comments and formatted output are used. Evidence is reflected in the test plan and testing document.

Strengths

1. Validation: blank, length and invalid-character inputs are handled before calculation, preventing misleading ratings.

2. Transparent scoring: every scoring component is printed, so the user can understand how the total was calculated.

3. Readable structure: `rate_password()` contains the scoring logic and `main()` handles interaction; meaningful names and consistent indentation improve maintainability.

4. Repeatable scoring: rules are implemented consistently and tested across varied input types.

Weaknesses and Limitations

1. The program is a text interface, which may be less accessible or intuitive than a graphical interface. A GUI with accessible labels could improve usability.

2. It rates a password but does not generate a new one. A secure random password generator could be added.

3. The program does not explain how to improve a weak password. Advice could be provided without revealing the password.

4. The scoring model is the supplied rubric and is not a complete modern security assessment (for example, it does not check password reuse or common-password lists).

Testing and Debugging

Twelve tests were run covering a mixed password, length bonus, minimum and maximum limits, too-short/too-long inputs, blank input, invalid characters and lowercase-only, uppercase-only and numeric-only passwords. All passed. Validation was designed to prevent scoring invalid inputs. See Activity 4 testing evidence for expected and actual results. The tests increased confidence that boundary and abnormal inputs are handled as required.

Code Quality

The code uses descriptive constants and variable names, consistent four-space indentation, a docstring for the main logic function and a separation between calculation and console presentation. This makes it easier to understand, test and modify. The character counts and scoring calculations are explicit rather than hidden in a complex expression.

Future Improvements

1. Add a secure password-generation feature using Python's `secrets` module, allowing users to create a strong candidate password.

2. Add a graphical interface and accessibility support, improving usability for a wider range of staff.

3. Add configurable scoring and guidance, while clearly distinguishing the provided assessment rubric from real-world password advice.

4. Add optional common-password checking locally, avoiding transmission of passwords to external services. Conclusion The solution meets the scenario's stated functional requirements: it accepts a password, validates length and character set, reports the scoring breakdown and total, and assigns a rating from the specified ranges. The test set passed and the code is structured in a readable and maintainable way. It is suitable for demonstrating the requested password-rating process, while the text interface and limited rubric remain areas for future development